**PROJECT REPORT**

**FOR**

**Foundations of Artificial Intelligence**

# AI Power Optimization

## Intelligent Household Backup Power Management using Simulated Annealing

**A PROJECT REPORT**


***Submitted by***

**2116240701604 – Vishnu Priyan M**

**2116240701560 - Thamizharasan M**





**October, 2026**

# BONAFIDE CERTIFICATE

Certified that this project report "**AI Power Optimization – Intelligent Household Backup Power Management using Simulated Annealing**" is the bonafide work of "**Vishnu Priyan M (240701604) & Thamizharasan M (240701560)**" who carried out the project work under my supervision.

**SIGNATURE OF THE FACULTY INCHARGE**

**Submitted for the Practical Examination held on _____**

**SIGNATURE OF THE INTERNAL EXAMINER**

## TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | ABSTRACT | 4 |
| 1 | INTRODUCTION | 5 |
| 2 | LITERATURE REVIEW | 8 |
| 3 | PROPOSED SYSTEM | 10 |
| | 3.1 ARCHITECTURE DIAGRAM | 14 |
| 4 | MODULES DESCRIPTION | 15 |
| | 4.1 MODULE 1 | 15 |
| | 4.2 MODULE 2 | 16 |
| 5 | IMPLEMENTATION AND RESULTS | 18 |
| | 5.1 EXPERIMENTAL SETUP | 18 |
| | 5.2 RESULTS | 20 |
| 6 | CONCLUSION AND FUTURE WORK | 25 |
| 7 | REFERENCES | 26 |

**ABSTRACT:**

The **AI Power Optimizer** is an intelligent energy-management system that decides how a household should use the limited backup power of an inverter and battery during a power outage. The available backup cannot run every appliance at the same time, so the system must decide which appliances are switched ON or OFF, in which time slots, and for how long, while protecting essential appliances such as the refrigerator and the Wi-Fi router.

The problem is formulated as a constrained combinatorial optimization problem. A candidate solution is a binary matrix *schedule[appliance][time_slot]* in which 1 means ON and 0 means OFF. For the demonstration scenario of six appliances and eight half-hour slots there are 2^48 possible schedules, so exhaustive search is impossible. The project therefore uses **Simulated Annealing (SA)**, a local-search algorithm from Unit II of Foundations of Artificial Intelligence. The algorithm was implemented from scratch in Python, without any optimization library, using six neighbour operators, a reward-and-penalty fitness function, the acceptance probability $P = \exp(\Delta E / T)$ and geometric cooling.

The system is delivered as a web application. A FastAPI backend validates the input with Pydantic models, runs the SA engine, builds a naive priority-based baseline for comparison and generates plain-language explanations and warnings. A browser dashboard built with HTML, CSS, JavaScript and Chart.js shows the optimized schedule as a Gantt chart, the battery-level and power-per-slot charts, a before-and-after comparison and the SA convergence curve. In the demonstration scenario (1000 W inverter, 3000 Wh battery at 70 %, four-hour outage) the optimizer keeps both essential appliances running for the whole outage, never exceeds the inverter limit, correctly refuses the 1200 W air conditioner and uses on average 1821 Wh of the 2100 Wh available. A suite of 36 automated tests verifies the implementation.

The work shows how a local-search algorithm can solve a practical, constraint-heavy scheduling problem while keeping every decision explainable. The appliance data and fitness weights are illustrative.

## 1. INTRODUCTION

Power outages are a common problem in many households. During an outage the house depends on a small backup system, usually an inverter with a battery or a UPS. Such a system has two hard limits: the **maximum power** the inverter can deliver at any instant, and the **total energy** stored in the battery. A refrigerator, a Wi-Fi router, a fan, a television, a water pump and an air conditioner together may demand far more than either limit allows. The household therefore has to choose what to run, when to run it and for how long.

Making this choice by hand is difficult, because every decision affects the others. Switching on a 750 W water pump leaves almost no room for other loads in that time slot; running the fan for too long may drain the battery before the refrigerator has met its minimum running time. The number of possible ON/OFF combinations grows exponentially with the number of appliances and time slots, so a systematic search of all combinations is not practical.

The **AI Power Optimizer** treats this situation as an Artificial Intelligence problem. An intelligent agent observes the environment (battery state, inverter capacity and appliance data), searches the space of possible schedules and acts by producing the schedule that gives the best overall utility. The search is performed by **Simulated Annealing**, a local-search optimization algorithm that starts from one schedule, repeatedly examines a slightly modified neighbour, and sometimes accepts a worse neighbour so that the search can escape local optima.

### Problem Statement

Given an inverter with a maximum output power, a battery with a known charge, an expected outage duration and a list of appliances with power ratings, priorities and duration requirements, find a schedule that decides **which appliances are ON or OFF, when each operates and for how long**, such that the inverter power limit is never exceeded, the battery energy is not exhausted, essential appliances receive at least their

minimum running time, and the most important appliances are served first.

## Objectives

- To model household backup-power scheduling as a constrained search and optimization problem.
- To implement Simulated Annealing from scratch, with clearly separated state, neighbour, fitness, constraint and cooling components.
- To guarantee protection of essential appliances and to respect inverter and battery limits through penalty terms in the fitness function.
- To compare the optimized schedule with a simple fixed-priority (naive) schedule.
- To present the result through an interactive dashboard with explanations that a non-technical user can follow.

## Mapping to the FOAI Syllabus

**Table 1.1: Mapping of project concepts to the FOAI syllabus**

| Syllabus concept | Where it appears in the project |
|---|---|
| **Intelligent agent, agent and environment (Unit I)** | The optimizer observes battery, inverter and appliance data (environment) and produces an appliance schedule (action). |
| **Rational behaviour, utility (Unit I)** | The agent maximizes a fitness (utility) function while respecting constraints. |
| **Problem formulation, state space (Unit II)** | State = binary matrix schedule[appliance][slot]; state space size 2^(appliances x slots). |
| **Local search, optimization (Unit II)** | Simulated Annealing keeps one current state and explores neighbours; no search tree is stored. |
| **Objective function / heuristic (Unit II)** | Fitness function with priority weights guiding the search. |
| **Escaping local optima (Unit II)** | Worse neighbours are accepted with probability exp(ΔE / T), which shrinks as T cools. |

Figure 1.1 shows the flow of the Simulated Annealing algorithm used in this project.

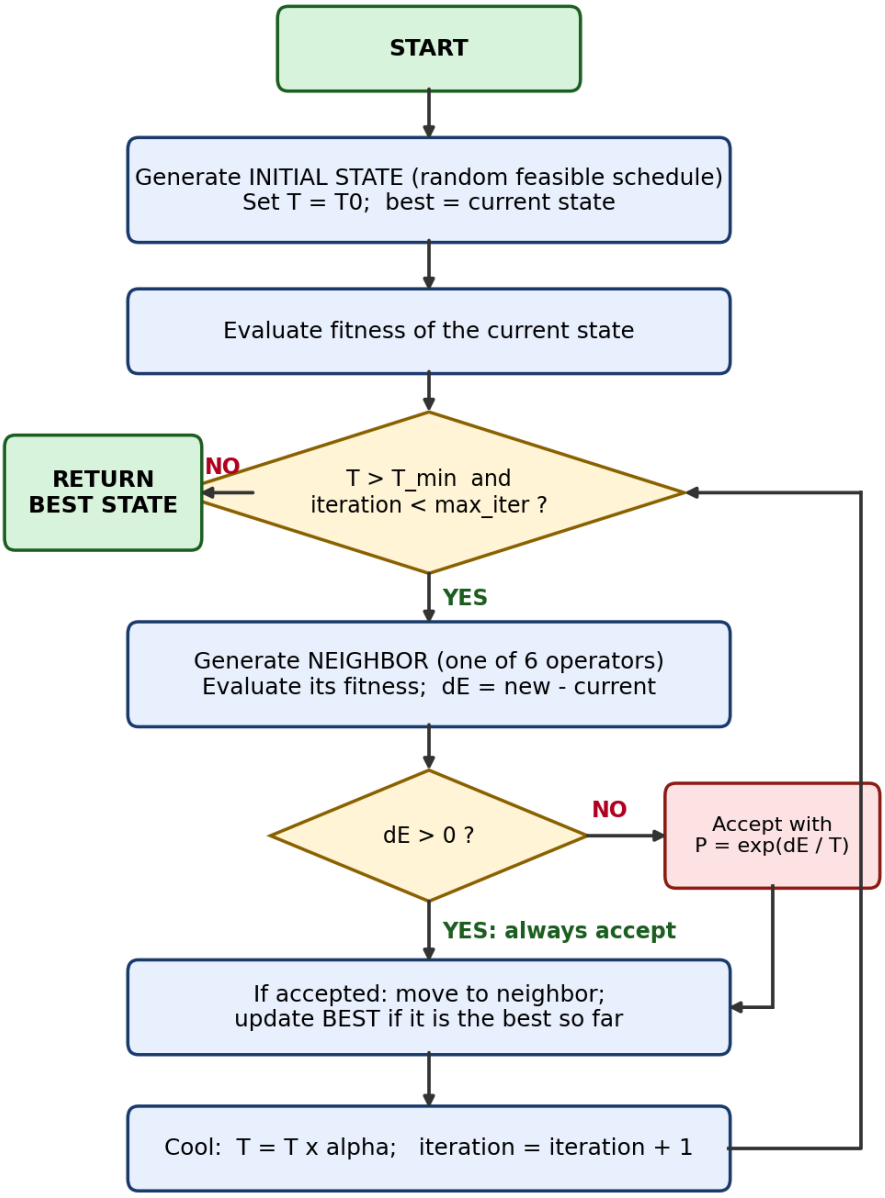



**Fig 1.1 Simulated Annealing Algorithm**

## 2. LITERATURE REVIEW

### Foundations of Simulated Annealing:

**Metropolis et al. (1953)** proposed a Monte Carlo procedure for simulating the behaviour of a collection of interacting particles at a given temperature. In their method a random change of the system is always accepted when it lowers the energy and is accepted with a probability that depends on the energy difference and the temperature when it raises the energy. This acceptance rule is the basis of the exp(ΔE / T) test used in the optimizer of this project.

**Kirkpatrick, Gelatt and Vecchi (1983)** recognised the analogy between the slow cooling of a physical material and the minimization of a cost function in a combinatorial problem, and introduced Simulated Annealing as a general optimization method. They applied it to problems such as the placement and wiring of computer circuits. Independently, **Černý (1985)** described a similar thermodynamic approach for the travelling salesman problem. These works justify the choice of SA for a discrete scheduling problem whose state space is too large to enumerate.

**Henderson, Jacobson and Johnson (2003)** reviewed the theory and practice of Simulated Annealing, discussing the effect of the cooling schedule, the neighbourhood structure and the stopping rule on the quality of the solution. Their discussion supports the parameters that the proposed system exposes to the user: the initial temperature, the minimum temperature, the cooling rate and the maximum number of iterations.

### Demand-Side Management and Household Appliance Scheduling:

**Palensky and Dietrich (2011)** surveyed demand-side management, demand response and smart loads and explained why shifting or reducing the consumption of household appliances is an important tool for energy systems. **Molderink et al. (2010)** studied the management and control of domestic smart-grid technology, in which the operation of household appliances and local energy resources is planned by software. **Setlhaolo, Xia**

**and Zhang (2014)** formulated the scheduling of household appliances as an optimization problem. These studies show that appliance scheduling is an established optimization task; the present project applies the idea to the special situation of an islanded backup supply, where the objective is to survive the outage rather than to reduce the electricity bill.

**Local Search and Problem Solving in Artificial Intelligence:**

**Russell and Norvig (2021)** present local search, hill climbing and Simulated Annealing as the standard techniques for optimization problems in which the path to the solution is irrelevant and only the final state matters. They also describe the formulation of problems through states, actions and objective functions and the concept of a rational agent. This project follows that formulation directly: the schedule is the state, the neighbour operators are the actions, and the fitness function is the objective.

**Research Gap and Contribution:**

The literature above treats Simulated Annealing and appliance scheduling separately and mostly in the context of grid-connected homes. The proposed system combines them for an outage scenario with a hard inverter limit, a finite battery and essential loads, implements the algorithm without external solvers, and adds a transparent explanation layer and a comparison with a naive schedule. This makes the algorithm easy to inspect, test and explain.

## 3. PROPOSED SYSTEM

The proposed system is the **AI Power Optimizer**, a web-based decision-support tool that produces an optimized appliance schedule for a power outage using **Simulated Annealing**. The user enters the backup-system data (inverter power, battery capacity, charge percentage and outage duration) and the list of appliances (power, priority, required, minimum and maximum duration, optional preferred time window and an essential flag). The outage is divided into equal time slots of 15, 30 or 60 minutes, and the system searches for the best ON/OFF pattern over these slots.

### Problem Formulation

**State.** A schedule is a two-dimensional binary matrix S in which S[i][t] = 1 if appliance i is ON in slot t. With n appliances and m slots, the state space has 2^(n x m) states.

**Initial state.** A random schedule that respects basic feasibility: appliances whose power is greater than the inverter limit are left OFF, essential appliances receive their required number of slots, and other appliances are switched on in randomly chosen slots with a probability that depends on their priority (0.8, 0.6, 0.4 and 0.2 for very high, high, medium and low priority).

**Neighbour state.** A schedule obtained by one small random change of the current schedule, produced by one of six operators (Table 3.2).

**Objective.** Maximize the fitness function of Table 3.1. Higher fitness means a better schedule.

**Constraints.** The five constraints of Table 3.3, enforced through large penalty terms.

**Simulated Annealing Algorithm**

```
current  <- generate_initial_solution()
best     <- current
T        <- T0                          # e.g. 1000
WHILE T > T_min AND iteration < max_iterations:
    neighbor <- generate_neighbor(current)      # one of 6 operators
    dE       <- fitness(neighbor) - fitness(current)
    IF dE > 0:   accept                          # better: always accept
    ELSE:        accept with probability P = exp(dE / T)
    IF accepted: current <- neighbor
                 IF fitness(current) > fitness(best): best <- current
    T         <- T * alpha                       # geometric cooling
    iteration <- iteration + 1
RETURN best
```

At a high temperature the factor exp(ΔE / T) is close to 1, so worse schedules are often accepted and the search explores widely. As the temperature falls, the factor approaches 0, so mostly improvements are accepted and the search settles near a good solution. The best schedule ever seen is stored separately, because the current state may wander to worse schedules during the search.

**Major Components**

1. **Input Interface**
    - Forms for backup system, appliance table and SA settings.
    - Default data loaded from the backend, with the option to add or edit appliances and to enable solar generation.
2. **Data Validation**
    - Pydantic models check ranges, enumerations and the rule that maximum duration is not smaller than minimum duration.
3. **Initial State Generator and Naive Baseline (scheduler.py)**
    - Creates the random feasible starting schedule.
    - Creates a fixed-priority greedy schedule used for comparison.
4. **Neighbour Generator (neighbor.py)**
    - Six mutation operators: toggle ON, toggle OFF, move, extend, reduce and swap.
5. **Fitness Evaluator (fitness.py)**

- Combines five rewards and four penalties into a single score.

6. **Constraint Checker (constraints.py)**
   - Computes slot power, total energy and per-constraint violations used in fitness evaluation and in the warnings.
7. **Simulated Annealing Engine (simulated_annealing.py)**
   - Main loop, acceptance rule, cooling schedule, best-solution tracking and telemetry for the charts.
8. **Explanation and Dashboard Layer**
   - Natural-language reason for every appliance, warnings for violated constraints, Gantt chart, battery and power charts, naive-versus-SA comparison and the convergence curve.

**Table 3.1: Components of the fitness function**

| Component | Formula | Weight / constant |
|---|---|---|
| **Priority reward** | sum( priority weight x slots ON x power / 100 ) | Very high 10, High 7, Medium 4, Low 1 |
| **Essential reward** | sum( 50 x slots ON ) for essential appliances | 50 per slot |
| **Duration satisfaction** | min( actual hours / required hours, 1 ) x 30 x priority weight | 30 |
| **Time preference** | ( slots in window / slots ON ) x 15 x priority weight | 15 |
| **Energy utilization** | Bonus when 60 to 90 % of available energy is used | 100 (ideal range) |
| **Power violation penalty** | - 10000 x (1 + excess ratio) per overloaded slot | - 10000 |
| **Energy violation penalty** | - 5000 x (1 + excess ratio) if energy exceeds battery | - 5000 |
| **Switching penalty** | - 2 x number of ON/OFF transitions | - 2 |
| **Essential violation penalty** | - 8000 x (1 + shortfall ratio) if below minimum duration | - 8000 |

**Table 3.2: Neighbour (mutation) operators**

| Operator | Effect on the schedule |
|---|---|
| **Toggle ON** | Sets a randomly chosen appliance ON in a randomly chosen slot. |
| **Toggle OFF** | Sets a randomly chosen appliance OFF in a randomly chosen slot. |
| **Move** | Moves one ON slot of an appliance to a different OFF slot. |
| **Extend** | Turns ON a slot adjacent to an existing ON slot, lengthening the run. |
| **Reduce** | Turns OFF one ON slot of an appliance, shortening the run. |
| **Swap** | Exchanges the ON/OFF values of two appliances within one slot. |

**Table 3.3: Constraints**

| No. | Constraint | Description | How it is handled |
|---|---|---|---|
| C1 | Power | Total power in every slot must not exceed the inverter maximum. | Penalty in fitness; appliances above the limit are never scheduled initially. |
| C2 | Energy | Total energy must not exceed the available battery energy (capacity x percentage). | Penalty in fitness. |
| C3 | Essential | Essential appliances must reach their minimum duration. | Penalty and per-slot bonus in fitness. |
| C4 | Time window | Appliances should run inside their preferred window. | Reward for in-window slots; checked after the run. |
| C5 | Maximum duration | No appliance should exceed its maximum duration. | Checked after the run and reported as a warning. |

**Table 3.4: Simulated Annealing parameters**

| Parameter | Symbol | Default (demo) | Purpose |
|---|---|---|---|
| Initial temperature | T0 | 1000 | Controls how freely worse schedules are accepted at the start. |
| Minimum temperature | T_min | 0.1 | Stopping condition. |
| Cooling rate | α | 0.95 | Temperature is multiplied by α after every iteration. |
| Maximum iterations | - | 10000 | Upper bound on the number of search steps. |
| Time slot | - | 30 min | Length of one schedule slot (15, 30 or 60 minutes). |

## 3.1 ARCHITECTURE DIAGRAM

The architecture has four layers. The browser interface collects the data; the FastAPI layer receives it as JSON and validates it with Pydantic models; the Simulated Annealing engine searches for the best schedule using the scheduler, neighbour generator, fitness evaluator and constraint checker; and the results are returned to the dashboard together with a naive baseline, explanations and warnings.

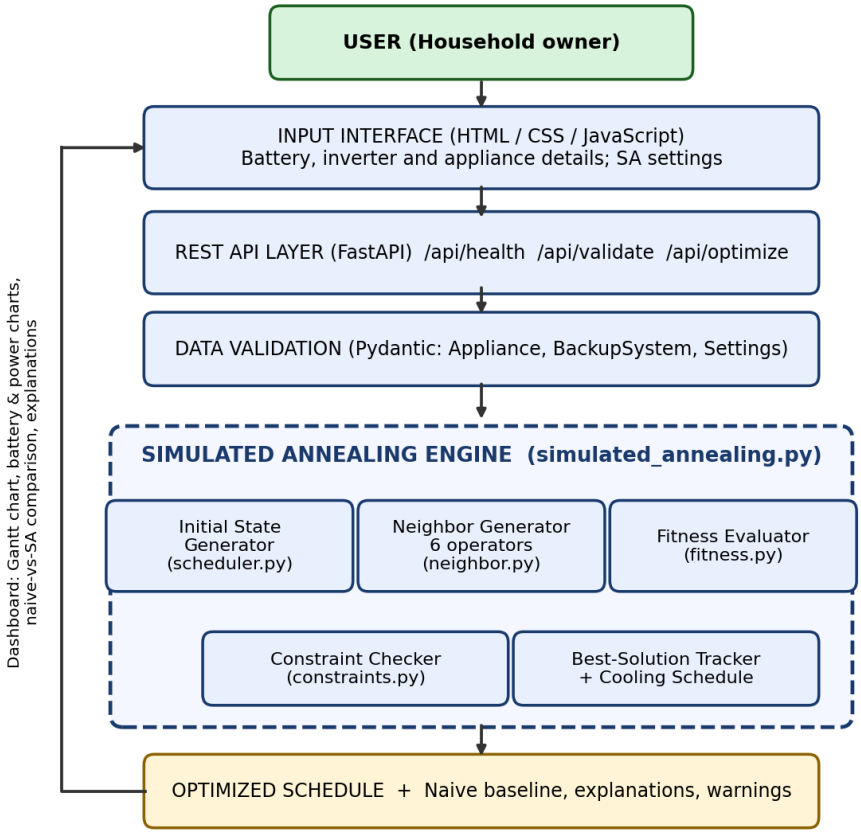



**Figure 3.1: System Architecture of the AI Power Optimizer**

## 4. MODULES DESCRIPTION

The AI Power Optimizer is organized into two major modules. The first module is the user interface that collects the household data and presents the results; the second module is the optimization and backend module that validates the data, runs Simulated Annealing and prepares the explanations. The two modules communicate through a small REST API.

### 4.1 MODULE 1 – USER INTERFACE AND VISUALIZATION MODULE

The user interface is a single-page dashboard written in HTML, CSS and JavaScript and served by the backend. It loads the default appliances and the demonstration scenario from the API and lets the user change any value. The backup card shows the maximum inverter power, battery capacity, battery percentage, outage duration, time-slot size and an optional solar switch, and it displays the available energy (capacity x percentage / 100) as the values are edited. The appliance table allows rows to be added, edited and removed, and the settings card exposes the SA parameters.

When the user presses the run button, the interface collects the values, shows an animated seven-step agent workflow (Observe, Analyze, Search, Optimize, Evaluate, Decide, Act), sends the request to */api/optimize* and renders the response. Results are shown as summary tiles, a Gantt-style schedule table, a battery-level line chart, a power-per-slot chart, a naive-versus-SA comparison chart, a list of explanations and the SA convergence curve. The charts are drawn with Chart.js.

**Main functions:**

- Entry and editing of backup-system data with live available-energy display.
- Appliance table with priority, duration limits, preferred time window and essential flag.
- Optional solar generation per slot.
- Configuration of SA parameters.

- Agent-workflow animation while the optimization runs.
- Summary tiles: fitness, energy consumed, remaining energy, final battery level, peak power, priority satisfaction, essential satisfaction and appliances powered.
- Gantt schedule, battery chart, power chart, comparison chart and convergence chart.
- Explanations and warnings for every appliance.

## 4.2 MODULE 2 – OPTIMIZATION AND BACKEND MODULE

The backend is written in Python with FastAPI. When a request reaches */api/optimize* the data is validated by the Pydantic models, the number of slots is calculated from the outage duration and the slot size, and the Simulated Annealing engine is started. The engine generates the initial state, evaluates its fitness and then runs the annealing loop described in Chapter 3, keeping the best schedule found and sampling the fitness and temperature roughly 200 times for the convergence chart.

After the search, the backend builds a naive baseline schedule in which appliances are sorted by essential flag and priority weight and are switched on greedily for their required duration while the power and energy limits allow. Both schedules are converted into metrics (energy consumed, remaining energy, peak power, priority satisfaction, essential satisfaction, number of appliances powered and estimated backup duration). Finally the backend generates one explanation per appliance from the real result and warnings for any violated constraint, such as an appliance whose rating is above the inverter limit.

The module is organized in files with one responsibility each: *simulated_annealing.py* (main loop), *fitness.py* (objective), *constraints.py* (constraint checks and energy calculations), *neighbor.py* (six operators), *scheduler.py* (initial and naive schedules), *appliance.py* (data models) and *app.py* (API).

**Table 4.1: REST API endpoints**

| Endpoint | Method | Description |
|---|---|---|
| /api/health | GET | Health check of the service. |
| /api/default-appliances | GET | Returns the default appliances and the demonstration scenario. |
| /api/validate | POST | Validates the input and reports problems without running the optimizer. |
| /api/optimize | POST | Runs Simulated Annealing and returns schedule, metrics, comparison, explanations and SA telemetry. |
| / | GET | Serves the dashboard. |

## Main functions:

- Request handling and input validation.
- Calculation of time slots, slot power, total energy, per-slot energy and battery percentage.
- Generation of the initial state and of the naive baseline.
- Neighbour generation with six operators.
- Fitness evaluation with rewards and penalties.
- Simulated Annealing loop with best-solution tracking.
- Computation of satisfaction metrics and comparison data.
- Generation of explanations and warnings.

## 5. IMPLEMENTATION AND RESULTS

The AI Power Optimizer was implemented as a web application. The backend uses Python and FastAPI; the optimizer is hand-written and uses only the Python standard library (math, random and copy). The frontend is plain HTML, CSS and JavaScript with Chart.js for the charts. No database is used, because all data is exchanged as JSON for each request. The source code is organized into a backend, a frontend and a tests folder, and the application is started with a single command that serves both the API and the dashboard on port 8000.

To evaluate the system, the demonstration scenario was run through the real API function and through the dashboard, additional scenarios with a smaller battery, a smaller inverter and a longer outage were executed, and the effect of the cooling rate was studied. Because Simulated Annealing is random, every experiment was repeated with several different random seeds.

### 5.1 EXPERIMENTAL SETUP

**Table 5.1: Technology stack and environment**

| Item | Details |
|---|---|
| **Language** | Python 3.13 (Python 3.8 or later is required) |
| **Backend** | FastAPI, Uvicorn, Pydantic v2 |
| **Optimization** | Hand-implemented Simulated Annealing (no OR-Tools, PuLP or SciPy optimize) |
| **Frontend** | HTML, CSS, JavaScript, Chart.js v4 |
| **Data** | JSON (default_appliances.json); no database |
| **Testing** | pytest: 36 tests in three files (constraints, fitness, optimizer) |
| **Platform** | Windows 11, Google Chrome |

**Table 5.2: Demonstration scenario**

| Parameter | Value |
|---|---|
| **Maximum inverter power** | 1000 W |
| **Battery capacity / charge** | 3000 Wh / 70 % (available energy 2100 Wh) |
| **Outage duration / slot size** | 4 hours / 30 minutes (8 slots, 2^48 possible schedules for 6 appliances) |
| **SA settings** | T0 = 1000, T_min = 0.1, α = 0.95, maximum 10000 iterations |

| Appliance | Power | Priority | Required / min / max (h) | Essential |
|---|---|---|---|---|
| Refrigerator | 150 W | Very high | 3.0 / 2.0 / 4.0 | Yes |
| Wi-Fi router | 10 W | High | 3.0 / 2.0 / 4.0 | Yes |
| Fan | 75 W | High | 2.0 / 1.0 / 4.0 | No |
| Television | 120 W | Medium | 1.0 / 0.5 / 3.0 | No |
| Water pump (06:00 to 09:00) | 750 W | Medium | 0.5 / 0.5 / 1.0 | No |
| Air conditioner | 1200 W | Low | 1.0 / 0.5 / 2.0 | No |

## Test scenarios

- Demonstration scenario: all appliances, 1000 W inverter, 70 % battery. Expected: the 1200 W air conditioner cannot run, both essential appliances run, no overload.
- Low battery (30 % and 15 %): available energy of 900 Wh and 450 Wh. Expected: low-priority loads are dropped first and essential loads are kept.
- Small inverter (800 W): the 750 W water pump can only run when few other appliances are on.
- Long outage (8 hours, 16 slots): larger search space with the same battery.
- Cooling-rate study: $\alpha = 0.90$, 0.95, 0.99 and 0.995.

The automated test suite (36 tests) checks slot-power and energy calculation, each of the five constraint checks, every component of the fitness function (for example, that a power violation lowers fitness by a large amount), the six neighbour operators, the generation of valid initial schedules and the behaviour of the full optimizer. All 36 tests pass.

## 5.2 RESULTS

Figure 5.2.1 shows the input screen of the dashboard with the demonstration data loaded. Figure 5.2.2 shows the summary tiles and the warning generated for the air conditioner after a typical run.

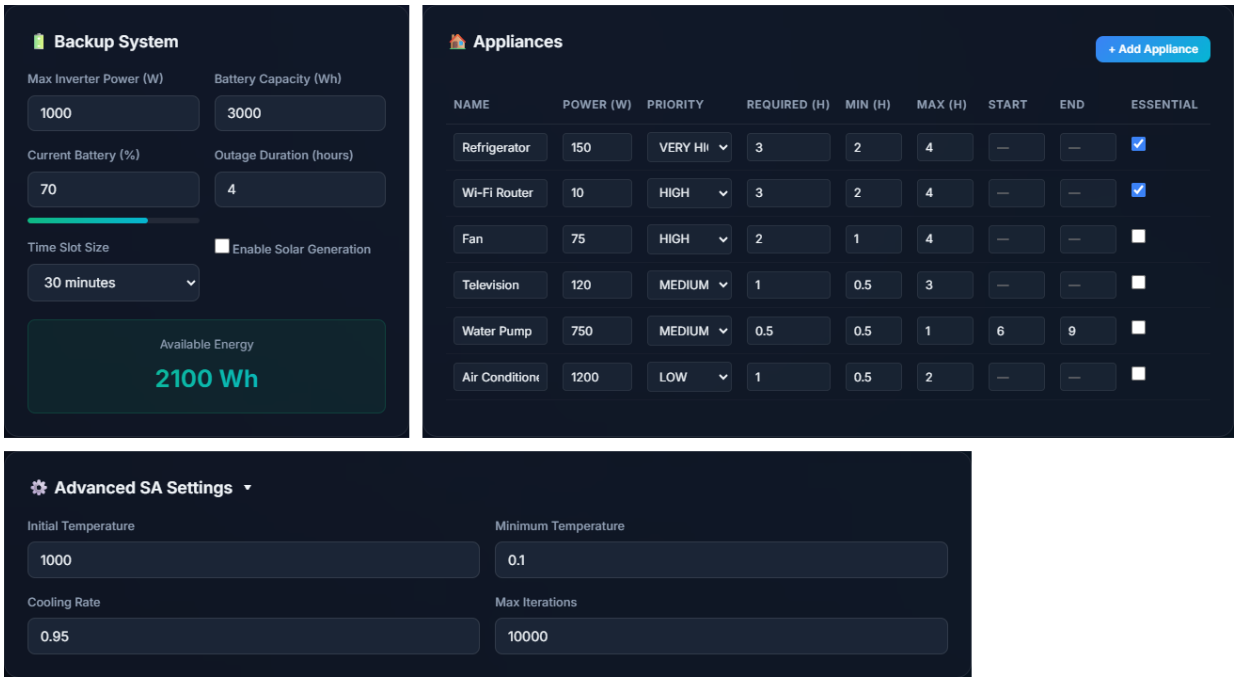



**Figure 5.2.1 Input Dashboard (Backup System, Appliances and SA Settings)**

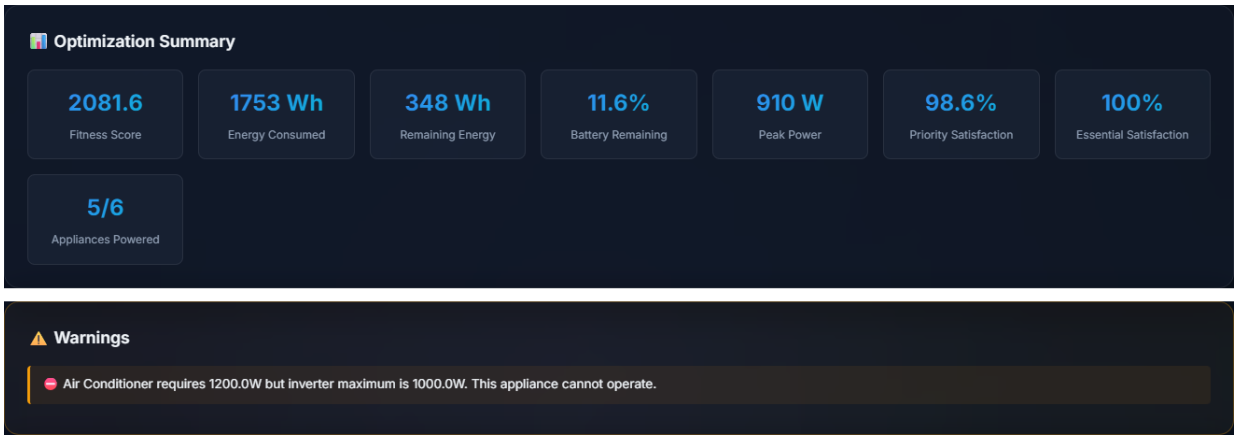



**Figure 5.2.2 Result Summary and Warning**

The optimized schedule of a typical run is shown as a Gantt-style table in Figure 5.2.3. The refrigerator and the Wi-Fi router are ON in every slot, the fan covers most of the outage, the television and the water pump are placed in a few slots, and the air conditioner is OFF in all slots because its 1200 W rating is greater than the 1000 W inverter limit. The total power row confirms that no slot exceeds 1000 W; the highest

values occur in the slots in which the 750 W water pump is running.




**Figure 5.2.3 Optimized Appliance Schedule**

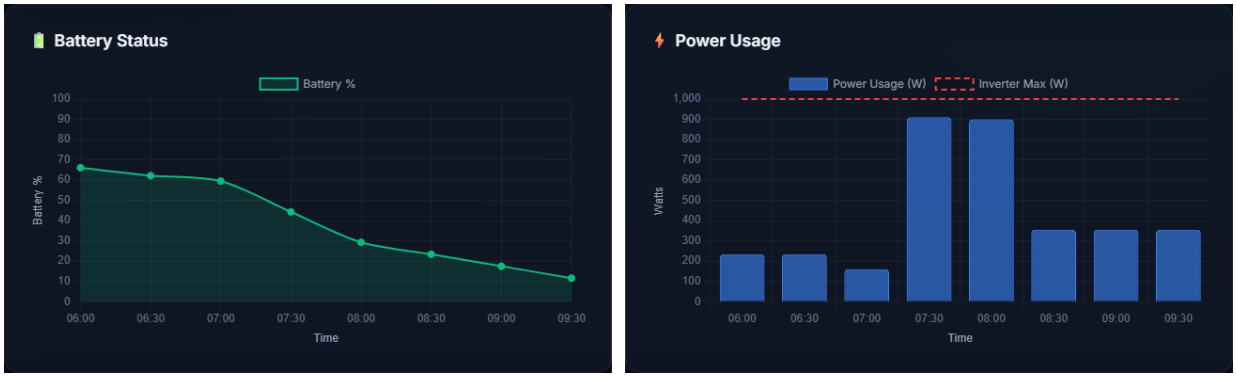



**Figure 5.2.4 Battery Level and Power per Slot**

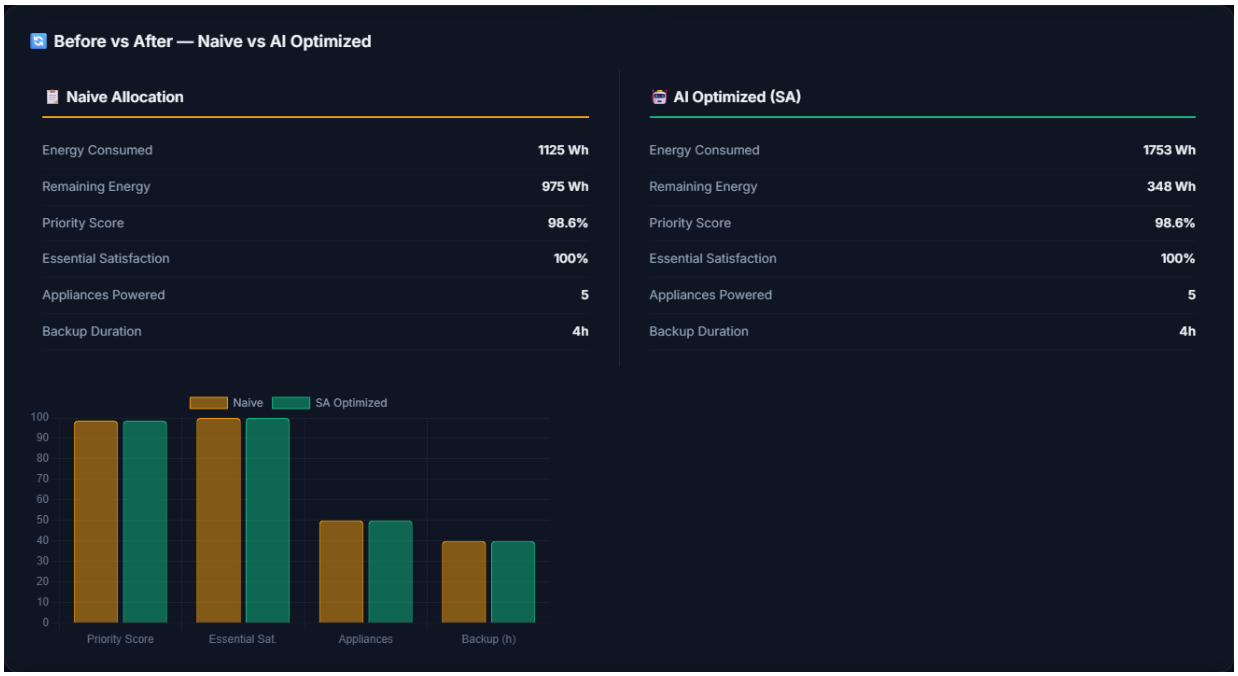



**Figure 5.2.5 Naive Allocation versus Simulated Annealing**




**Figure 5.2.6 Explanation for Every Appliance**




**Figure 5.2.7 Simulated Annealing Details and Convergence Curve**

## Results for the demonstration scenario

The demonstration scenario was run 30 times with different random seeds. Table 5.3 gives the mean, minimum and maximum of the main metrics. In every run both essential appliances were fully satisfied (100 %), five of the six appliances were powered (the air conditioner never), and the peak power stayed below the 1000 W limit. The energy consumed was on average 1821 Wh of the 2100 Wh available, which lies in the 60 to 90 % band that the fitness function rewards (about 87 %). With $\alpha = 0.95$ the loop stops after 180 iterations, because the temperature falls below T_min before the limit of 10000 iterations is reached; on average 54 % of the neighbours were accepted.

**Table 5.3: Demonstration scenario, 30 runs**

| Metric | Mean | Minimum | Maximum |
|---|---|---|---|
| **Final fitness** | 2043.5 | 1888.5 | 2156.0 |
| **Energy consumed (Wh)** | 1820.6 | 1470.0 | 1885.0 |
| **Remaining energy (Wh)** | 279.4 | 215.0 | 630.0 |
| **Peak power (W)** | 971.3 | 825.0 | 985.0 |
| **Priority satisfaction (%)** | 98.2 | 91.7 | 98.6 |
| **Essential satisfaction (%)** | 100.0 | 100.0 | 100.0 |
| **Appliances powered (of 6)** | 5 | 5 | 5 |

## Results for other scenarios

Table 5.4 compares the Simulated Annealing result with the naive baseline in five scenarios (10 runs each for SA; the naive schedule is deterministic). In all scenarios the energy consumed stays within the available battery energy, the peak power stays within the inverter limit and the essential appliances are fully satisfied.

**Table 5.4: Scenario comparison (SA = mean of 10 runs)**

| Scenario | Available (Wh) | Inverter (W) | SA energy (Wh) | SA peak (W) | SA priority (%) | Naive priority (%) | Essential (%) |
|---|---|---|---|---|---|---|---|
| **Demonstration** | 2100 | 1000 | 1830 | 985 | 98.6 | 98.6 | 100 |
| **Battery 30 %** | 900 | 1000 | 840 | 985 | 87.4 | 95.8 | 100 |
| **Battery 15 %** | 450 | 1000 | 416 | 280 | 65.1 | 41.7 | 100 |
| **Inverter 800 W** | 2100 | 800 | 1422 | 760 | 97.8 | 95.8 | 100 |
| **Outage 8 hours** | 2100 | 1000 | 1880 | 985 | 98.6 | 98.6 | 100 |

The results show that Simulated Annealing is clearly better than the naive schedule when the battery is very low (65.1 % against 41.7 % priority satisfaction at 15 % charge) and slightly better with a small inverter (97.8 % against 95.8 %). In the demonstration scenario the priority satisfaction is equal (98.6 %), but the naive schedule leaves 975 Wh unused whereas the optimized schedule uses far more of the battery and still keeps a reserve. At a 30 % battery, however, the naive schedule reaches a higher priority score (95.8 % against 87.4 %). This happens because the fitness function is not identical to the priority-satisfaction metric: the fitness rewards priority weight multiplied by power and a 60 to 90 % energy-utilization band, whereas the metric counts hours weighted by priority regardless of power. The result is reported as obtained; it indicates that the fitness

weights could be tuned further for low-battery situations.

**Effect of the cooling rate**

Table 5.5 shows the effect of the cooling rate on the solution quality over 20 runs each. A faster cooling rate ends the search after fewer iterations and gives lower and more variable fitness, whereas a slower cooling rate takes more iterations and gives higher and more stable fitness. For $\alpha = 0.995$ the minimum fitness (2144.3) is close to the maximum (2158.0), which shows that the search converges reliably to near-identical solutions. The number of iterations equals $\ln(T_{min} / T0) / \ln(\alpha)$ rounded up, so the limit of 10000 iterations is only reached for cooling rates very close to 1.

**Table 5.5: Effect of the cooling rate (20 runs each)**

| Cooling rate α | Iterations | Mean fitness | Minimum | Maximum |
|---|---|---|---|---|
| 0.9 | 88 | 1986.0 | 1786.3 | 2132.2 |
| 0.95 | 180 | 2047.5 | 1912.8 | 2156.0 |
| 0.99 | 917 | 2128.4 | 2022.2 | 2156.0 |
| 0.995 | 1838 | 2152.4 | 2144.3 | 2158.0 |

## Discussion

- **Constraint handling:** hard constraints are enforced as large penalties. In all experiments the final schedules respected the inverter and battery limits, and the essential appliances always received their minimum duration.
- **Explainability:** every appliance has a generated reason (for example, "OFF for the entire outage because its power consumption (1200 W) exceeds the inverter capacity (1000 W)"), so the user can see why the schedule looks as it does.
- **Randomness:** because the method is stochastic, results vary slightly between runs; the best fitness in 30 runs ranged from 1888.5 to 2156.0 with the default $\alpha = 0.95$, and a slower cooling rate reduces this spread.
- **Limitations:** the maximum-duration constraint (C5) is checked and reported after the search but is not part of the fitness function; the outage is assumed to start at 06:00; and the appliance data and weights are illustrative.

## 6. CONCLUSION AND FUTURE WORK

The AI Power Optimizer shows how Simulated Annealing, a local-search algorithm from FOAI Unit II, can be applied to a practical problem: scheduling household appliances on a limited inverter and battery during a power outage. The problem was formulated with a binary schedule matrix as state, six neighbour operators, a fitness function that rewards priority, essential operation, duration satisfaction, time preference and good energy use, and penalizes power, energy and essential-appliance violations. The algorithm was implemented from scratch and tested with 36 automated tests.

In the experiments the optimizer always kept the essential appliances running, respected the inverter and battery limits, refused appliances that could never run on the inverter and, in constrained situations such as a very low battery, produced a better priority score than a naive fixed-priority schedule. The study of the cooling rate showed the expected trade-off between running time and solution quality. The dashboard and the generated explanations make the result understandable to a non-technical user. The experiments also revealed a limitation: with a 30 % battery the naive schedule reached a higher priority score, so the fitness weights need further tuning.

### FUTURE WORK

The system can be extended with real-time battery monitoring, so that the schedule is recomputed as the battery state changes, and with multiple backup sources such as grid, generator and solar. Weather-based solar prediction and multi-day outage planning would make the energy model more realistic. The maximum-duration constraint can be added to the fitness function, the outage start time can be made configurable, and the fitness weights can be calibrated or learned from real appliance usage. A mobile interface and a comparison of Simulated Annealing with hill climbing, genetic algorithms and exact methods would complete the study.

# 7. REFERENCES


1. Metropolis, N., Rosenbluth, A.W., Rosenbluth, M.N., Teller, A.H. and Teller, E. (1953) 'Equation of State Calculations by Fast Computing Machines', *The Journal of Chemical Physics*, Vol. 21, No. 6, pp. 1087–1092.

2. Kirkpatrick, S., Gelatt, C.D. and Vecchi, M.P. (1983) 'Optimization by Simulated Annealing', *Science*, Vol. 220, No. 4598, pp. 671–680.

3. Černý, V. (1985) 'Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm', *Journal of Optimization Theory and Applications*, Vol. 45, No. 1, pp. 41–51.

4. Henderson, D., Jacobson, S.H. and Johnson, A.W. (2003) 'The Theory and Practice of Simulated Annealing', in Glover, F. and Kochenberger, G.A. (eds) *Handbook of Metaheuristics*. Boston: Kluwer Academic Publishers, pp. 287–319.

5. Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*, 4th edn. Hoboken: Pearson.

6. Palensky, P. and Dietrich, D. (2011) 'Demand Side Management: Demand Response, Intelligent Energy Systems, and Smart Loads', *IEEE Transactions on Industrial Informatics*, Vol. 7, No. 3, pp. 381–388.

7. Molderink, A., Bakker, V., Bosman, M.G.C., Hurink, J.L. and Smit, G.J.M. (2010) 'Management and Control of Domestic Smart Grid Technology', *IEEE Transactions on Smart Grid*, Vol. 1, No. 2, pp. 109–119.

8. Setlhaolo, D., Xia, X. and Zhang, J. (2014) 'Optimal Scheduling of Household Appliances for Demand Response', *Electric Power Systems Research*, Vol. 116, pp. 24–28.

9. Python Software Foundation (2025) 'Python Documentation'. Available at: https://docs.python.org/3/

10. FastAPI (2025) 'FastAPI Documentation'. Available at: https://fastapi.tiangolo.com/

11. Pydantic (2025) 'Pydantic Documentation'. Available at: https://docs.pydantic.dev/

12. Chart.js Contributors (2025) 'Chart.js Documentation'. Available at: https://www.chartjs.org/docs/